Nathan Oliver
CSCI 403
11/16/22
Lab6

2.1

$$\pi_{id,\ source,\ destination}(uber\_report\ ur \times weather\_report\ wr)$$

$$= uber\_report \bowtie_{\substack{source = location \\ rain > 0.09 \\ date(wr.time\_stamp) = date(ur.time\_stamp)}} weather\_report$$

2.2

```
                                                                    QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------
 Unique  (cost=2048.06..2051.81 rows=375 width=68) (actual time=729.713..867.496 rows=28945 loops=1)
   ->  Sort  (cost=2048.06..2049.00 rows=375 width=68) (actual time=729.712..787.808 rows=455807 loops=1)
         Sort Key: ur.id, ur.source, ur.destination
         Sort Method: external merge  Disk: 18864kB
         ->  Merge Join  (cost=1754.46..2032.03 rows=375 width=68) (actual time=99.093..344.356 rows=455807 loops=1)
               Merge Cond: ((ur.source = wr.location) AND (((ur.time_stamp)::date) = ((wr.time_stamp)::date)))
               ->  Sort  (cost=1587.59..1624.14 rows=14620 width=138) (actual time=96.841..105.874 rows=30491 loops=1)
                     Sort Key: ur.source, ((ur.time_stamp)::date)
                     Sort Method: external merge  Disk: 2240kB
                     ->  Seq Scan on uber_report ur  (cost=0.00..576.20 rows=14620 width=138) (actual time=0.030..55.083 rows=30491 loops=1)
               ->  Sort  (cost=166.87..169.44 rows=1027 width=102) (actual time=2.192..54.857 rows=455822 loops=1)
                     Sort Key: wr.location, ((wr.time_stamp)::date)
                     Sort Method: quicksort  Memory: 42kB
                     ->  Seq Scan on weather_report wr  (cost=0.00..115.50 rows=1027 width=102) (actual time=0.025..1.961 rows=204 loops=1)
                           Filter: (rain > 0.09)
                           Rows Removed by Filter: 6072
 Planning Time: 0.222 ms
 Execution Time: 872.650 ms
(18 rows)
```

a) The column selection happens first, sorted with an external merge. First filter occurs with another merge on source and location, and then there are two separate sorts for both the date qualification and the rain condition.
b) There do not appear to be any indexes. Only sorts after merges, and then sequence scans.
c) The cross product wasn't an entire merge of both tables. There were separate merges on conditions. First being source = location (merge join), then two separate sorts for time stamps (external merge). The sort for time stamp in weather_report had an additional scan for rain > 0.09 (quicksort).
d) It used an external merge join to verify uniqueness (this was the first executable in the query plan)

2.3

```
                                                                QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------
 HashAggregate  (cost=6065.84..6081.79 rows=1595 width=68) (actual time=653.255..658.252 rows=28945 loops=1)
   Group Key: ur.id, ur.source, ur.destination
   Batches: 1  Memory Usage: 3857kB
   ->  Merge Join  (cost=5467.73..6053.88 rows=1595 width=68) (actual time=98.116..359.934 rows=455807 loops=1)
         Merge Cond: ((ur.source = wr.location) AND (((ur.time_stamp)::date) = ((wr.time_stamp)::date)))
         ->  Sort  (cost=5196.89..5273.12 rows=30491 width=138) (actual time=95.962..105.783 rows=30491 loops=1)
               Sort Key: ur.source, ((ur.time_stamp)::date)
               Sort Method: external merge  Disk: 2240kB
               ->  Seq Scan on uber_report ur  (cost=0.00..734.91 rows=30491 width=138) (actual time=0.027..55.398 rows=30491 loops=1)
         ->  Sort  (cost=270.83..276.06 rows=2092 width=102) (actual time=2.095..58.670 rows=455822 loops=1)
               Sort Key: wr.location, ((wr.time_stamp)::date)
               Sort Method: quicksort  Memory: 42kB
               ->  Seq Scan on weather_report wr  (cost=0.00..155.45 rows=2092 width=102) (actual time=0.022..1.864 rows=204 loops=1)
                     Filter: (rain > 0.09)
                     Rows Removed by Filter: 6072
 Planning Time: 0.964 ms
 Execution Time: 660.071 ms
(17 rows)
```

e) Because the query began by scanning both tables entirely, I made an index of the primary keys for each table. This way the initial scan is cut down significantly before the merge join.

f) The query now scans the hash created by the indexes, instead of scanning both entire tables. It also does it only once. The rest of the query more or less proceeds the same. The new run time has now improved by about 33%!

2.4

```
                                                                QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------
 HashAggregate  (cost=1276.43..1352.66 rows=7623 width=68)
   Group Key: uber_report.id, uber_report.source, uber_report.destination
   ->  Hash Join  (cost=181.33..1219.25 rows=7623 width=68)
         Hash Cond: ((uber_report.source = weather_report.location) AND ((uber_report.time_stamp)::date = (weather_report.time_stamp)::date))
         ->  Seq Scan on uber_report  (cost=0.00..734.91 rows=30491 width=138)
         ->  Hash  (cost=172.08..172.08 rows=617 width=102)
               ->  HashAggregate  (cost=165.91..172.08 rows=617 width=102)
                     Group Key: weather_report.location, (weather_report.time_stamp)::date
                     ->  Seq Scan on weather_report  (cost=0.00..155.45 rows=2092 width=102)
                           Filter: (rain > 0.09)
(10 rows)
```

g) Instead of the search having the appearance of a having binary tree, we now have a much more linear path. It appears that even though it runs the same number of scans, it runs more efficiently. This time, it begins with a hash join with a prior condition, and then multiple hash scans afterwards. This way, almost all of the work is being done on the index, not the actual tables. Once the hash scans have selected all the relevant information, there is still a basic scan to select the entries with rain > 0.09.